

Milestone 2 Feature Report

Nouns Builder — before & after of the four Milestone 2 feature tracks

Funding context	Gnars DAO (Base) — Proposal 61: “Enhance the Official Builder Template and Upstream Feature Batches”, Milestone 2 of 3
Repository	BuilderOSS/nouns-builder (monorepo)
Branch	feat/m2-feature-tracks — 5 commits over base c4a90bf (branch staging)
Changeset	54 files changed, +3,287 / -22 lines
Date	June 12, 2026
Team	r4topunk (implementation) · Vlad (@gnarlyvlad)

Executive summary

Milestone 2 delivers the four “lower-complexity” feature tracks committed in Proposal 61: a state-aware **voting power explanation** on proposal pages, **vote metrics visualizations** (quorum progress, vote timeline, participation history), **active member detection** in the members list, and **time-based urgency alerts** for the activity feed and dashboard. All four were implemented as reusable packages in the Nouns Builder monorepo with unit and component tests, no new runtime dependencies, no subgraph schema changes, and are bundled into a single upstream pull request.

HOW THE SCREENSHOTS WERE PRODUCED

Both versions of the app were run locally against production data: **Before** is the staging branch (commit c4a90bf), **After** is the feat/m2-feature-tracks branch. Because no Builder DAO had a proposal in its live voting window at capture time, on-chain reads for Base were pointed at an Anvil fork of Base mainnet at block 45,445,327 — a block inside Gnars Proposal 61’s actual voting window — so the proposal page renders its genuine “Active” voting state. The voting-power-explanation states were captured with mock-connected wallets using real Gnars member addresses, so every value shown (vote weights, delegates) comes from live on-chain reads. The urgency-alerts capture renders the shipped component with demonstration data, since the alerts only appear when a real deadline is near.

Voting Power Explanation

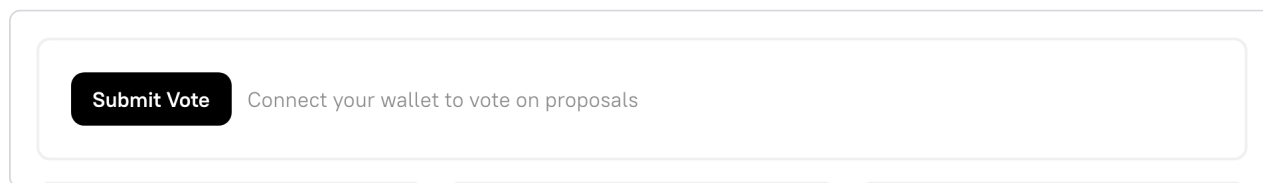
What changed. The vote panel on proposal pages now renders a `VotingPowerExplainer` that tells the viewer, in plain language, what their voting power is *for this specific proposal* and what they can do about it. Previously the panel showed a single terse message (“You must hold at least one token to vote”) regardless of why the viewer couldn’t vote.

Why. Voting power in Builder DAOs is snapshot-based: what counts is the delegation state at the proposal’s creation, not current balances. This is the single most common source of governance confusion — especially for members who acquired a token *after* the snapshot and can’t understand why they’re unable to vote.

How it works. A pure, unit-tested decision tree (`getVotingPowerCase`) maps connection, delegation, ownership and snapshot state to one of six cases, each with specific copy and a contextual call-to-action:

State	What the viewer sees
Not connected	Prompt to connect a wallet to see their voting power
No tokens	How voting power works + link to the live auction
Tokens acquired after snapshot	Clear explanation of checkpoint voting with the snapshot date
Delegated away	Who currently holds their votes + link to update delegation
Delegate for others	Their aggregated voting power (own + delegated)
Can vote / already voted	Their vote weight, or their submitted vote and its weight

BEFORE LEGACY VOTE PANEL



The entire legacy guidance on Gnars Proposal 61: a “Submit Vote” button and one generic line, regardless of the viewer’s actual situation. Connected viewers without usable voting power got a similarly terse “You must hold at least one token to vote”.

AFTER VOTINGPOWEREXPLAINER — STATE-AWARE GUIDANCE FOR CONNECTED VIEWERS

CAN VOTE — 0XCEED...D915 (42 VOTES)

You have **42 votes** available for Gnars

Connected as 0xCEED...D915, a Gnars delegate: “You have 42 votes available for Gnars” — the 42 votes are the address’s real voting power at the Proposal 61 snapshot, read on-chain. The Submit Vote button renders alongside in the full vote panel.

NO TOKENS — 0X1111...1111 (0 TOKENS)

You do not hold any Gnars tokens, so you cannot vote on this proposal. Win a token at auction to vote on future proposals.

[Bid in the current auction](#)

Connected as an address holding no Gnars: the explainer says why voting is unavailable and routes the viewer to the live auction with a “Bid in the current auction” link.

DELEGATED AWAY — 0X05E1...2EDD → 0XDcf3...7A19

Your 35 votes are delegated to 0xdcf...97a19 for this proposal.

[Update delegation](#)

Connected as 0x05E1...2EDD, which owns 35 Gnars currently delegated to 0xdcf...97a19: the explainer names the current delegate and offers an “Update delegation” link.

These states were captured with mock-connected wallets using real Gnars member addresses on Proposal 61 — the weights and delegate addresses shown are live on-chain values, not fabricated data. A viewer with no wallet connected still sees the standard connect prompt, unchanged.

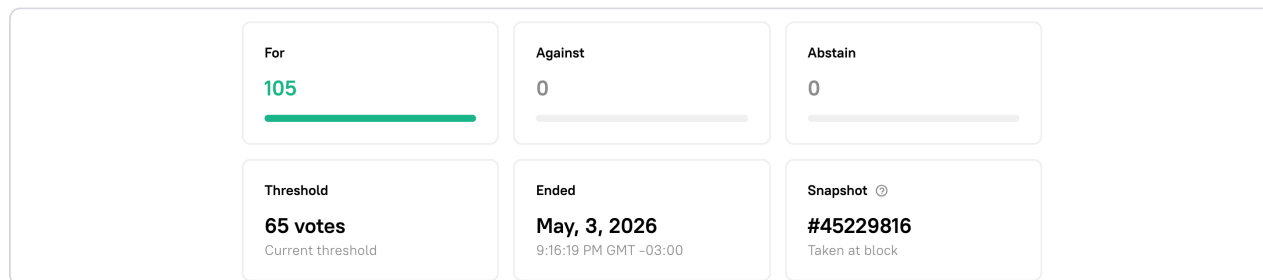
Vote Metrics & Visualizations

What changed. Proposal pages gain three visual metrics, all rendered as dependency-free custom SVG following the codebase's existing chart pattern: `QuorumProgress` (accumulated FOR votes against the quorum threshold, with a marker), `VoteTimeline` (cumulative for/against/abstain votes across the voting window, with hover detail), and `ParticipationHistory` (participation rate across the DAO's last ten proposals).

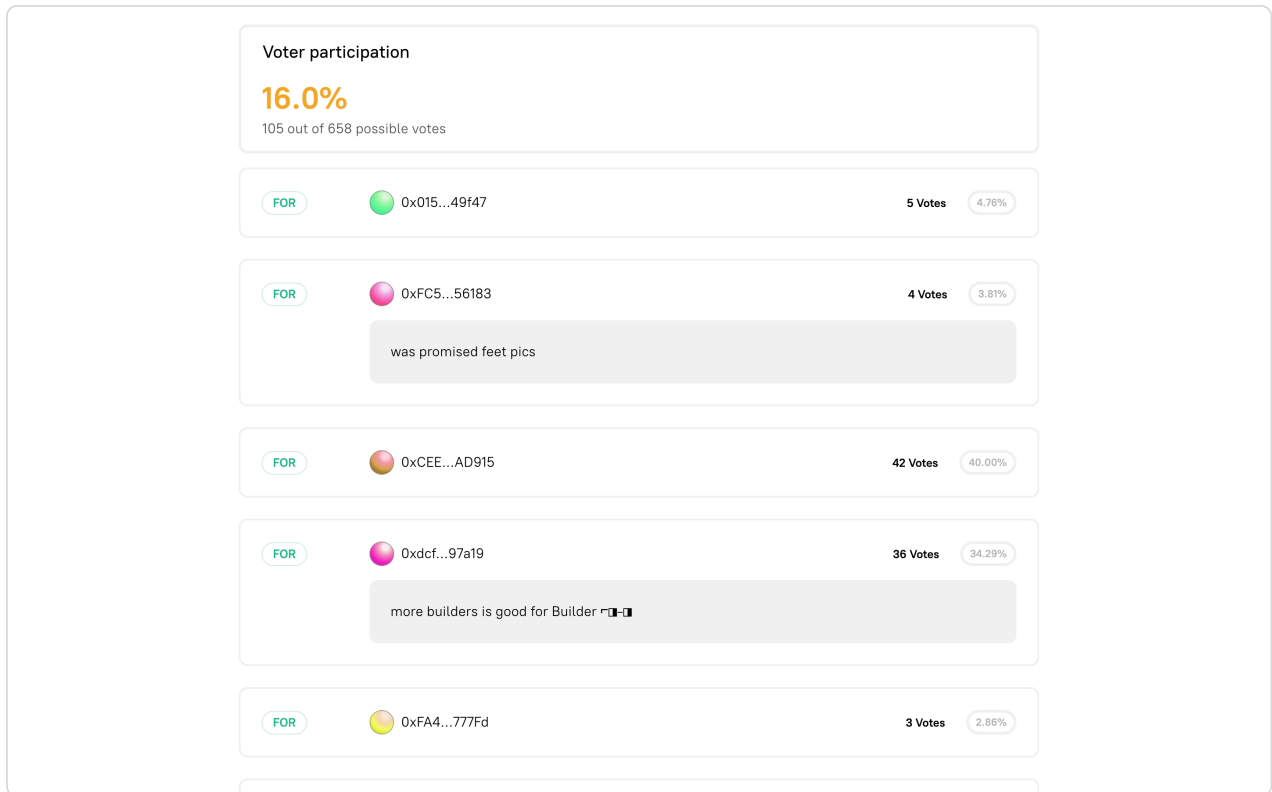
Why. Voters and proposers previously had to parse raw tallies and the voter table to answer basic questions: Is this proposal on track to meet quorum? When did momentum arrive? Is participation trending up or down?

How it works. Chart math lives in a fully unit-tested helper; the components consume vote data already indexed by the subgraph (a `timestamp` field was added to the existing `ProposalVote` GraphQL fragment — an additive, backward-compatible query change, not a schema change). `QuorumProgress` renders inside the proposal details grid; the two charts render in the Votes tab.

BEFORE VOTE TALLY TILES ONLY

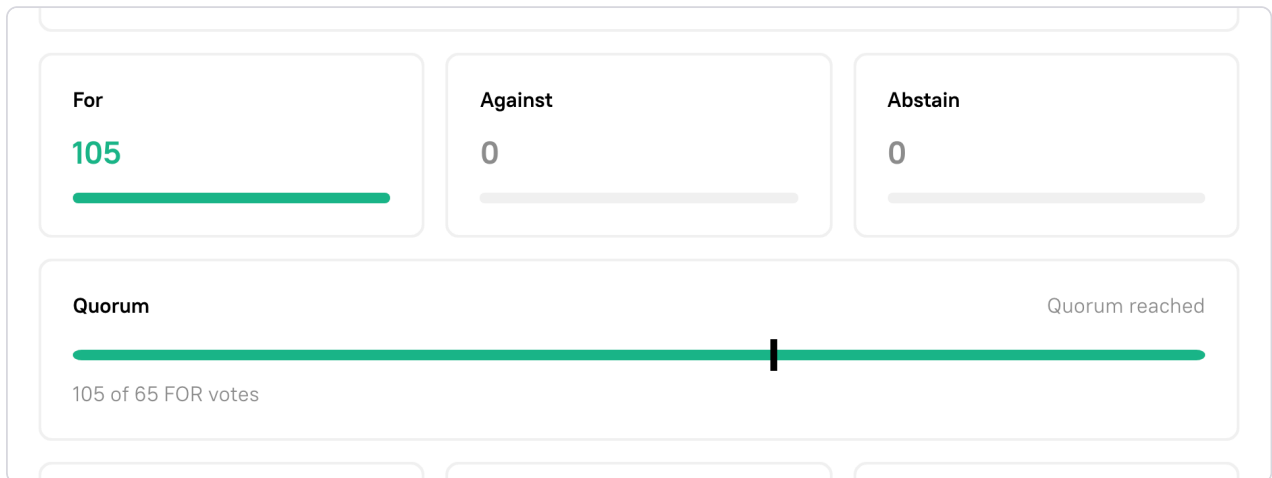


Baseline details grid: For (105) / Against (0) / Abstain (0) tiles with plain progress bars, plus the Threshold (65 votes), Ended and Snapshot tiles. Quorum exists only as a static number.



Baseline Votes tab: the voter participation summary (16.0% — 105 of 658 possible votes) followed directly by the flat voter list. No visual narrative of how or when the votes arrived.

AFTER QUORUM PROGRESS, VOTE TIMELINE AND PARTICIPATION HISTORY



The same details grid with the new QuorumProgress bar rendered beneath the tiles: accumulated FOR votes against the quorum marker, with the “Quorum reached — 105 of 65 FOR votes” readout.

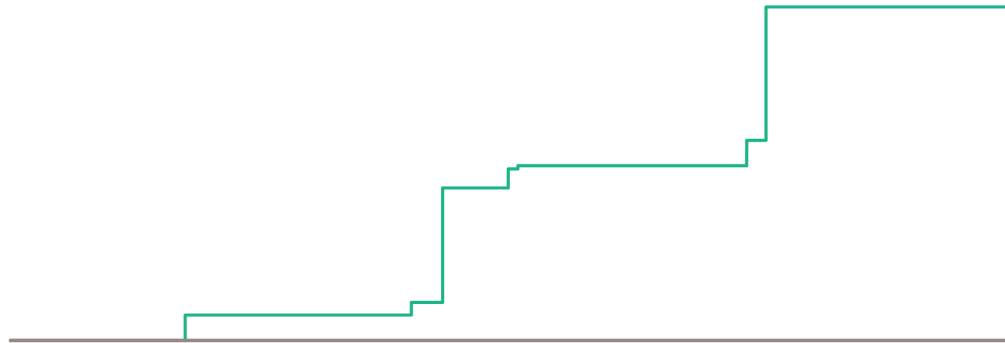
16.0%

105 out of 658 possible votes

Voting timeline

● For 105 ● Against 0 ● Abstain 0

May 3, 9:16 PM



Apr 29, 2026, 9:16 PM

May 3, 2026, 9:16 PM

FOR

0x015...49f47

5 Votes

4.76%

FOR

0xFC5...56183

4 Votes

3.81%

was promised feet pics

FOR

0xCee...AD915

42 Votes

40.00%

FOR

0xdcf...97a19

36 Votes

34.29%

more builders is good for Builder 🏠-🏠

FOR

0xFA4...777Fd

3 Votes

2.86%

FOR

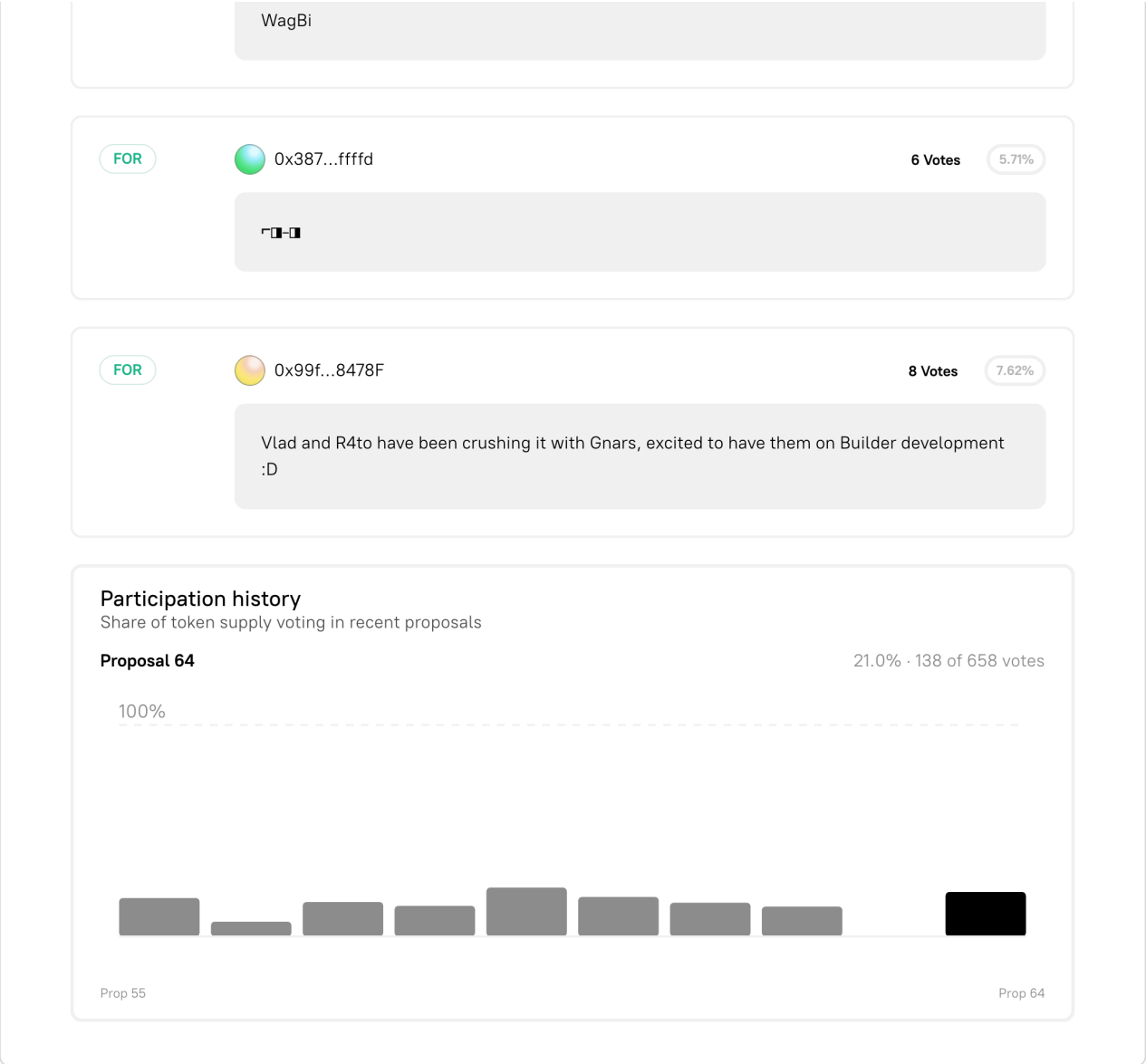
0x8Bf...a3C26

1 Vote

0.95%

Thanks for trusting us the honor of building along side with such skilled builders from builder dao.

Lets Build



Active Member Detection

What changed. The members list now distinguishes engaged members from passive holders: an **Active** badge on member cards and an **All / Active** filter (with counts) in the list header.

Why. Token ownership says nothing about engagement. For delegation choices, treasury decisions and outreach, DAOs need to see at a glance who actually participates.

How it works. Activity is computed entirely client/API-side from data the subgraph already indexes — three queries total for the whole list (recent proposals, their votes, recent auction bids), not one per member. **Definition (configurable, pending DAO validation):** a member is *active* if they voted on at least one of the DAO’s last 5 completed proposals *or* placed an auction bid in the last 30 days. While activity data loads (or on error) the UI degrades gracefully: badges hide and the filter disables, leaving the original list untouched.

BEFORE MEMBERS LIST WITHOUT ENGAGEMENT SIGNAL

Delegates

Export CSV

Delegate	Votes	Vote %	Joined
 0xFA2...2CcB8	65 Tokens	9.88%	Mar 22, 2024
 0xCCE...AD915	42 Tokens	6.38%	Mar 22, 2024
 0x1cD...Eef21	39 Tokens	5.93%	Mar 22, 2024
 0xdcf...97a19	36 Tokens	5.47%	Mar 22, 2024
 0x4bF...3bCc0	31 Tokens	4.71%	Mar 22, 2024
 0x74f...e07B4	26 Tokens	3.95%	Mar 22, 2024

[Documentation](#) [Proposal Guidelines](#) [Privacy policy](#) [Terms of service](#)

Baseline Delegates list: address, vote count, vote share and join date per member — no signal of who actually participates in governance.

AFTER ACTIVE BADGES AND ALL/ACTIVE FILTER

Delegates

[All \(176\)](#)[Active \(18\)](#)[Export CSV](#)

Delegate	Votes	Vote %	Joined
 0xFA2...2CcB8	65 Tokens	9.88%	Mar 22, 2024
 0xCEE...AD915 Active	42 Tokens	6.38%	Mar 22, 2024
 0x1cD...Eef21	39 Tokens	5.93%	Mar 22, 2024
 0xdcf...97a19 Active	36 Tokens	5.47%	Mar 22, 2024
 0x4bF...3bCc0 Active	31 Tokens	4.71%	Mar 22, 2024
 0x74f...e07B4	26 Tokens	3.95%	Mar 22, 2024

The same list with the new All (176) / Active (18) filter in the header and green “Active” badges on members who voted in recent proposals or bid in the last 30 days — computed live from subgraph data at render time.

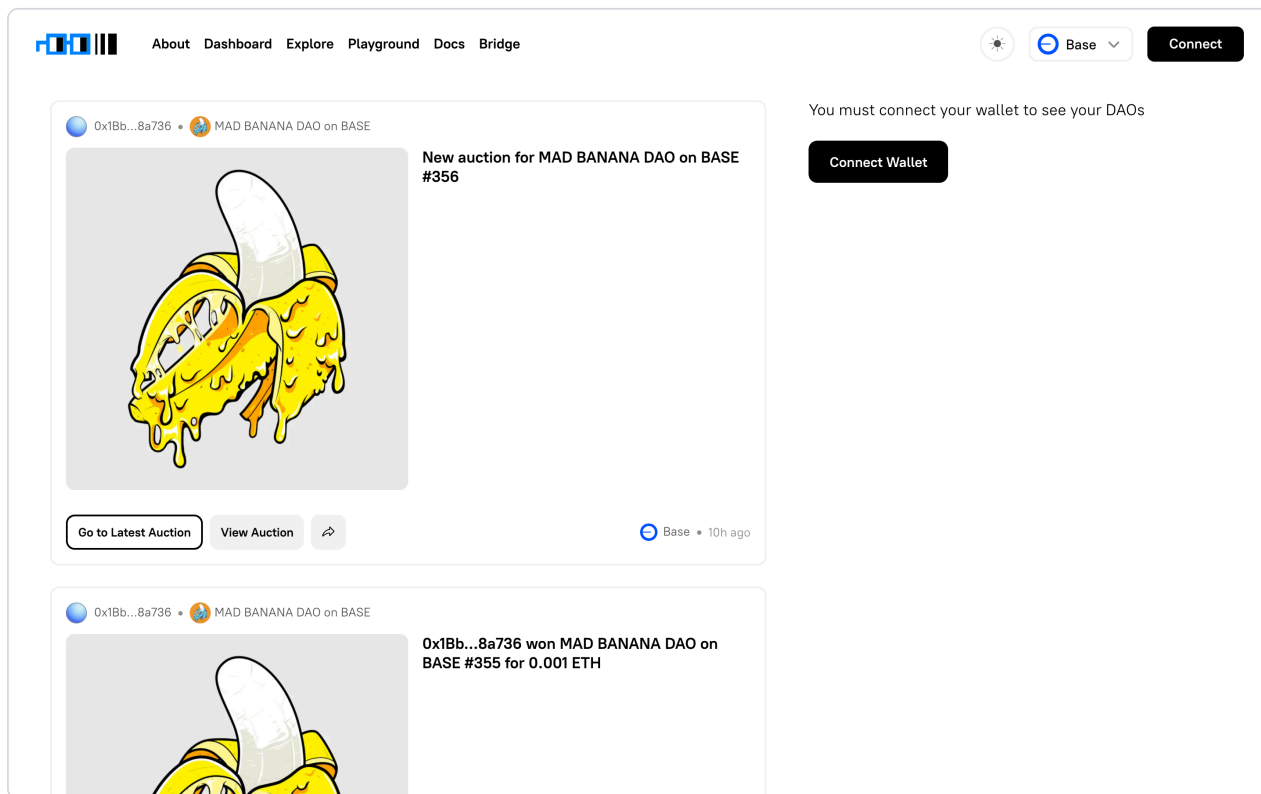
Time-Based Feed Alerts

What changed. The dashboard now pins an `UrgencyAlerts` strip above the activity feed, surfacing time-critical windows across the user's DAOs: an auction about to end, an active vote about to close, and an execution window about to expire. Each alert carries a live countdown, a deep link to the resource, and a session-scoped dismiss action.

Why. The feed is chronological — it tells members what already happened, not what is about to expire. Members routinely miss votes and execution windows because nothing foregrounds time pressure.

How it works. Alerts are derived client-side from data the dashboard already fetches — no new subgraph events, no backend. **Thresholds (configurable, pending DAO validation):** warning under 24 hours, critical under 2 hours, with distinct visual treatment. Alerts recompute on a 30-second interval and when a countdown ends.

BEFORE DASHBOARD FEED WITHOUT URGENCY SURFACING



Baseline dashboard: a purely chronological activity feed. Nothing distinguishes an auction ending in an hour from one that ended yesterday.

AFTER URGENCYALERTS (RENDERED WITH DEMONSTRATION DATA)

**Auction ending soon**

Gnars DAO · Gnars #766

01h 29m 56s

**Voting ending soon**

You haven't voted

Gnars DAO · Enhance the Official Builder Template and Upstream Feature Batches

19h 59m 56s

**Execution window expiring**

Gnars DAO · Fund the Q3 community grants round

29h 59m 56s



The shipped UrgencyAlerts items: a critical alert (red, <2 h) for an auction ending in 1 h 29 m, a warning alert (amber, <24 h) for a closing vote — including the “You haven’t voted” badge — and a warning for an expiring execution window. Each has a live countdown, a deep link and a dismiss control.

The capture above renders the shipped component with demonstration data (an auction ending in ~1.5 h, a vote closing in ~20 h, an execution window expiring in ~30 h), since alerts only appear when a real deadline is near. In production it renders at the top of the dashboard feed for the connected wallet’s DAOs.

Verification

Check	Result
pnpm build:packages	PASS — 19/19 tasks
pnpm type-check	PASS — 39/39 tasks
New test suites	All green — feed-ui 26/26 · dao-ui 14/14 · hooks 24/24 · utils 126/126 · proposal-ui 58 passing (incl. all new VotingPowerExplainer and VoteMetrics tests)
Pre-existing failures	6 tests fail identically on the base branch (<code>storage.getItem</code> in shared test fixtures) — verified unrelated to this changeset
Dependencies	No new runtime dependencies; only test tooling (devDependencies) added to feed-ui, mirroring the existing auction-ui setup
Per-track review	Each track was implemented on an isolated branch and passed an adversarial code review before integration

Every component ships with colocated tests; thresholds for active-member detection and urgency alerts are parameterized so the DAO can tune them without code changes.